



Fundamentos de Aprendizaje Automático 2024/2025

PRÁCTICA Nº 2: REGRESIÓN LOGÍSTICA, ANÁLISIS ROC y K-MEANS

1. Objetivo

En esta práctica continuaremos el estudio de los clasificadores trabajando con el algoritmo de *Regresión Logística*. Una vez implementado este método realizaremos un estudio comparativo de los clasificadores de los que disponemos hasta el momento: Naive-Bayes, Vecinos Próximos y Regresión Logística. El estudio se realizará empleando la técnica del Análisis ROC. Finalmente haremos una breve incursión en el aprendizaje no supervisado (clustering) mediante la implementación del algoritmo k-Means.

2. Conjuntos de datos

Regresión Logística

Con vistas al Análisis ROC posterior utilizaremos los conjuntos de datos *heart* y *wdbc* que ya se usaron en la práctica uno. De esta forma, los tres algoritmos se evaluarán sobre los mismos datasets.

K-means

El conjunto de patrones a usar en este algoritmo consiste en el conocido dataset *Iris* que clasifica tres tipos de flores en base a cuatro parámetros numéricos. El dataset se proporciona en Moodle.

3. Regresión Logística

3.1. Implementación propia

La implementación propia seguirá el algoritmo del descenso de gradiente iterativo que maximiza la verosimilitud estudiado en las clases de teoría. En la fase de **entrenamiento** se iterará sobre el conjunto de train y se realizará el ajuste de los pesos según dicho algoritmo durante un número determinado de épocas. Por su parte, en la fase de **clasificación**, los pesos calculados se mantendrán fijos y servirán para asignar las probabilidades a las instancias del conjunto de test y poder predecir la clase correspondiente. En esta implementación propia solo consideraremos el caso de uso de dos clases, es decir, no implementaremos una regresión logística multinomial.



3.2.Scikit-Learn

Scikit-learn proporciona una implementación de la *Regresión Logística* a través de los métodos **LogisticRegression** (https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html) y **SGDClassifier** (https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html). Revisa la información sobre estos métodos para comprender bien las diferencias entre la implementación propia y la que hace Scikit Learn de este algoritmo.

4. Evaluación de hipótesis mediante Análisis ROC

En la implementación de los clasificadores hemos utilizado como medida de evaluación la precisión del mismo, es decir, el porcentaje de aciertos/errores en el conjunto de test. Esta medida, que se utiliza frecuentemente por ser muy simple, tiene el inconveniente de que asume que todos los errores tienen igual importancia, algo que generalmente no ocurre, siendo poco apropiada por tanto en caso de clases desbalanceadas. Podemos encontrar medidas más adecuadas para evaluar los clasificadores y entre ellas destaca el *Análisis ROC* (Receiver Operating Characteristic). El análisis ROC muestra el rendimiento de un clasificador medido como Tasa de Verdaderos Positivos (TPR, en eje Y) frente a la Tasa de Falsos Positivos (FPR, en eje X). TPR frente a FPR es lo que se denomina Espacio ROC.

Para calcular esto, en primer lugar obtenemos la **matriz de confusión** para los conjuntos de datos **heart** y **wdbc**. Hay una matriz por cada conjunto y clasificador.

ESTIMADO	REAL	
	True Positives (TP)	False Positives (FP)
	False Negatives (FN)	True Negatives (TN)

A partir de estos valores obtenemos las tasas correspondientes:

$$TPR = TP / (TP + FN)$$

$$FNR = FN / (TP + FN)$$

$$FPR = FP / (FP + TN)$$

$$TNR = TN / (FP + TN)$$

Con estos valores dados por la matriz de confusión podemos dibujar un punto en el Espacio ROC. Si el clasificador devuelve solo la predicción de la clase, como ocurre con nuestra implementación de los algoritmos, solo podremos dibujar un punto en dicho espacio ROC. Se aconseja dibujar **dos espacios**, uno por dataset, con un punto por cada clasificador y espacio. A partir de estos puntos podemos evaluar cada algoritmo de clasificación teniendo en cuenta su posición respecto a la diagonal, que



representa una clasificación completamente aleatoria. **Modifica las clases donde creas oportuno para crear la matriz de confusión en nuestra suite de clasificadores y poder analizar el Espacio ROC.** En la parte de Scikit-Learn puedes utilizar esta implementación o llamar al método *confusión_matrix* (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html).

Para dibujar la curva ROC característica de un clasificador necesitamos conocer los umbrales de clasificación del modelo. Es decir, conocer la confianza que tiene un clasificador en una clasificación. Por ejemplo, en Regresión Logística podemos usar la probabilidad de pertenecer a la clase C1, en lugar de la etiqueta final. La asignación de una etiqueta se realiza si la probabilidad de pertenecer a la clase es mayor que un determinado valor umbral. Una forma de calcular la Curva ROC para la Regresión Logística sería ir variando los umbrales para obtener distintos puntos en el Espacio ROC y unirlos posteriormente. Un método alternativo más eficiente que no implica ejecutar continuamente la Regresión Logística consiste en devolver un *score* para cada instancia de test, en este caso la propia probabilidad de pertenencia a la clase calculada por la regresión. Después de ordenar los scores de mayor a menor vamos ascendiendo desde el punto (0,0) hacia arriba en el Espacio ROC, teniendo en cuenta que nos movemos en vertical si la instancia pertenece realmente a la clase C1 (positiva) y nos movemos hacia la derecha si la instancia pertenece realmente a la clase C2 (negativa). El destino será el punto (1,1). De esta forma obtenemos los puntos de la Curva ROC. Podemos etiquetar los puntos con el score que lo produce. Este score es el umbral. Una vez calculada la curva, el área bajo la misma se conoce como AUC (*Area Under the Curve*). El estudio de esta curva también ofrece información de interés sobre el rendimiento del clasificador. **Para aplicar esta técnica se debe añadir un método en el Clasificador Regresión Logística que devuelva los scores de las instancias de test.**

5. Algoritmo k-means

5.1. Implementación propia

K-means es un algoritmo de clustering particional, esto es, un algoritmo que inicialmente agrupa un conjunto de patrones en un número dado (K) de subconjuntos llamados clústeres, y va reasignando algunos de los patrones de un clúster a otros clústeres hasta que se cumple cierta condición de finalización, que de algún modo refleja la “calidad” de la agrupación de patrones alcanzada.

De manera más específica, dado un número K de clústeres, el algoritmo consta de los siguientes pasos:

1. Selecciona aleatoriamente k patrones (semillas) que serán los “centroides” iniciales de los clústeres.
2. Asigna cada uno del resto de patrones al centroide más cercano (atendiendo a una métrica de distancia dada).
3. En cada clúster elige un nuevo centroide de entre los patrones miembros del clúster, usando un determinado método de cálculo de centroides. Por ejemplo,



puedes usar como nuevo centroide el patrón medio del clúster.

4. Comprueba si se alcanza cierto criterio de convergencia. En caso de que no se alcance vuelve al paso 2.

Para medir las distancias entre puntos se pueden emplear diferentes métricas, entre las que se pueden destacar las siguientes:

- Distancia Euclídea

$$dist(\mathbf{x}_i, \mathbf{m}_j) = \|\mathbf{x}_i - \mathbf{m}_j\| = \sqrt{(x_{i1} - m_{j1})^2 + (x_{i2} - m_{j2})^2 + \dots + (x_{ir} - m_{jr})^2}$$

- Distancia Euclídea ponderada

$$dist(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{w_1(x_{i1} - x_{j1})^2 + w_2(x_{i2} - x_{j2})^2 + \dots + w_r(x_{ir} - x_{jr})^2}$$

- Distancia Manhattan

$$dist(\mathbf{x}_i, \mathbf{x}_j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ir} - x_{jr}|$$

- Distancia Cheychev

$$dist(\mathbf{x}_i, \mathbf{x}_j) = \max(|x_{i1} - x_{j1}|, |x_{i2} - x_{j2}|, \dots, |x_{ir} - x_{jr}|)$$

A su vez, existen varios criterios de convergencia para detener el algoritmo de clustering. Se pueden destacar los siguientes:

- Ningún o un número mínimo de re-asignaciones de patrones entre clusters.
- Ningún o un número mínimo de cambios de centroides.
- Una disminución mínima de la suma del error (distancia) cuadrático, del inglés *Sum of Squared Error* (SSE):

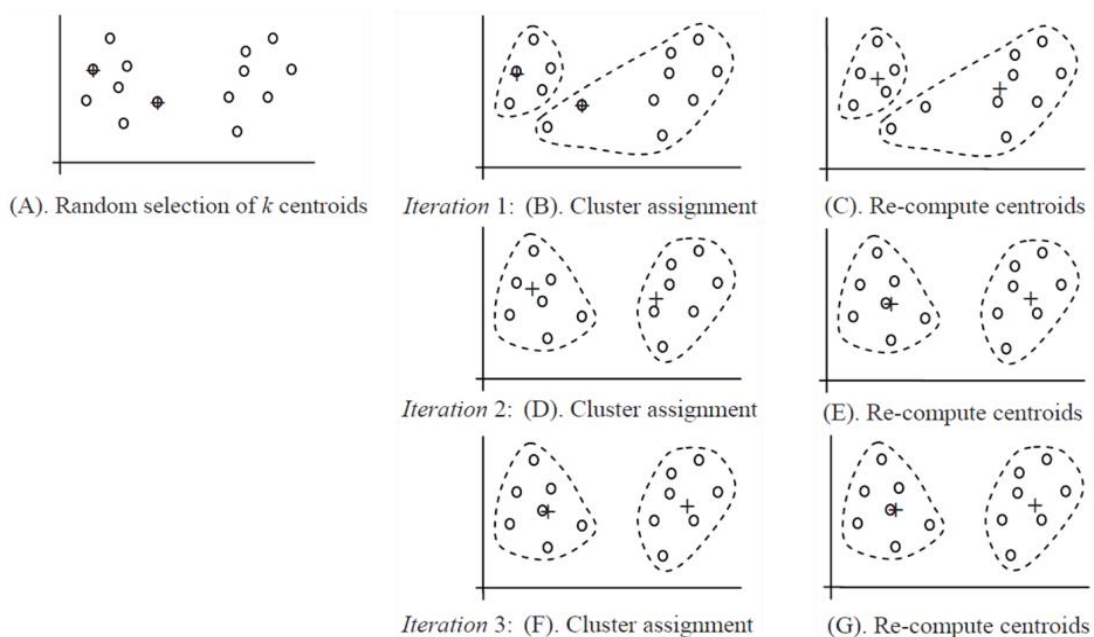
$$SSE = \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} dist(\mathbf{x}, \mathbf{m}_j)^2 \quad \mathbf{m}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i$$

donde C_j es el j -ésimo cluster, $\mathbf{m}_j$ es el centroide del cluster C_j (el vector media de todos los patrones de C_j), y $dist(\mathbf{x}, \mathbf{m}_j)$ es la distancia entre el patrón $\mathbf{x}$ y el centroide $\mathbf{m}_j$.

La siguiente figura muestra el pseudocódigo del algoritmo K-means, para un valor entero k de clusters y un conjunto de patrones D .

Algorithm k -means(k, D)
 choose k data points as the initial centroids (cluster centers)
repeat
 for each data point $\mathbf{x} \in D$ **do**
 compute the distance from $\mathbf{x}$ to each centroid;
 assign $\mathbf{x}$ to the closest centroid // a centroid represents a cluster
endfor
 re-compute the centroid using the current cluster memberships
until the stopping criterion is met

Las siguientes figuras, por su parte, ejemplifican la evolución del algoritmo para $k=2$.



5.2. Scikit-Learn

En Scikit-learn se encuentra el método **KMeans** (<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>) que implementa el clustering con este algoritmo. Es interesante repasar la guía de usuario (<https://scikit-learn.org/stable/modules/clustering.html#k-means>) en la que está basado el desarrollo.

6. Tareas

La planificación temporal sugerida y las tareas a llevar a cabo son las siguientes:

- *1ª semana*: Implementar el algoritmo de *Regresión Logística* (clase **ClasificadorRegresionLogistica**), aplicando el método de maximización de la verosimilitud visto en las clases de teoría. Analizar los resultados obtenidos para los conjuntos de datos **heart** y **wdbc**. Realizar el mismo análisis empleando la librería de Scikit-learn para este algoritmo, según los detalles del



apartado 3.2. Considera datos estandarizados y sin estandarizar.

- *2ª semana:* Realizar un estudio comparativo del rendimiento de los tres clasificadores (Naive-Bayes, K-nn y Regresión Logística) mediante un Análisis ROC en los dos conjuntos de datos considerados. Como hay muchas ejecuciones utiliza para este análisis solo el mejor resultado obtenido con cada clasificador. Para aplicar el Análisis ROC se aconseja seguir los detalles del apartado 4. **IMPORTANTE:** Ten en cuenta que el Análisis ROC explicado es de dos tipos: Espacio ROC y Curva ROC. Para el Espacio ROC considera los tres algoritmos y para la Curva ROC trabaja solo con la Regresión Logística.
- *3ª semana:* Implementar y evaluar el algoritmo K-means para realizar una tarea de clustering del dataset *Iris*. Para ello habrá que ejecutarlo primero con $K = 3$ (presumiblemente asignados a los tres tipos de flor) y comprobar cuántos patrones de cada clase (flor) quedan asignados a cada clúster. En este sentido, se pretende:
 - Comprobar si se puede asignar de forma unívoca cada clúster a un tipo de flor atendiendo a la clase mayoritaria de los patrones agrupados por clúster.
 - Analizar qué tipos de flor se identifican más fácilmente (con menos errores), cuales más difícilmente (con más errores), y cuales se confunden entre ellos. Para ello se puede hacer uso de la matriz de confusión multiclase a partir del número de puntos en cada clase y sabiendo que hay 50 ejemplos de cada tipo de flor.
 - Probar con otros valores de $K = [2,4,5]$ y comparar los resultados con los obtenidos para $k=3$. ¿Cuál sería el mejor K para este problema?
 - Analizar si cambiarían los resultados estandarizando los datos

IMPORTANTE: K-means no es un método de clasificación, por lo que nuestro diseño no aplica en este caso. K-means se debe presentar en un fichero .py distinto con todo lo que consideres necesario para su ejecución.

Realiza el mismo estudio de K-means empleando la librería de Scikit-learn en el mismo dataset Iris, de acuerdo con los detalles del apartado 5.2.

7. Fecha de entrega y entregables

21 de Noviembre de 2024. La entrega debe realizarse antes del comienzo de la clase de prácticas correspondiente. Se deberá entregar un fichero comprimido .zip con nombre **FAAP2_<grupo>_<pareja>.zip** (ejemplo FAAP2_1461_1.zip) y el siguiente contenido:

1. **Ipython Notebook (.ipynb)** con las instrucciones necesarias para realizar las pruebas descritas en los apartados anteriores y el correspondiente análisis de resultados. El Notebook debe estructurarse para contener los siguientes apartados:



Apartado 1	Resultados de la clasificación mediante regresión logística en los conjuntos de datos heart y wdbc. Probar con dos o tres valores para la constante de aprendizaje y el número de pasos.
Apartado 2	Resultados de la clasificación utilizando los algoritmos de Scikit-Learn para regresión logística. Comparación con los resultados de la implementación propia. En los resultados utilizar los mismos valores de constante de aprendizaje y pasos que en la implementación propia.
Apartado 3	Aplicar el análisis ROC para comparar los clasificadores Naive Bayes, Vecinos Próximos y Regresión Logística en los conjuntos heart y wdbc: • Dibuja los Espacios ROC de cada uno de estos conjuntos, incluyendo los tres clasificadores en cada gráfica. Discute los resultados. • Para la Curva ROC utiliza solo el clasificador de Regresión Logística basándote en el método comentado en el apartado 4 que comienza con la ordenación descendente de los scores. Analiza el área bajo la curva (AUC) y discute los resultados encada uno de los dos conjuntos de datos.
Apartado 4	Análisis descrito en la sección de Tareas para el algoritmo de clustering <i>K-means</i>, es decir: ○ Para $K=3$, comprobar si se puede asignar de forma unívoca cada clúster a un tipo de flor atendiendo a la clase mayoritaria de los patrones agrupados por clúster ○ Analizar qué tipos de flor se identifican más fácilmente y cuales se confunden entre ellos. Para ello se puede hacer uso de la matriz de confusión multiclase a partir del número de puntos en cada clase y sabiendo que hay 50 ejemplos de cada tipo de flor ○ Probar con otros valores de $K=[2,4,5]$ y comparar los resultados con los obtenidos para $K=3$. ¿Cuál sería el mejor K para este problema? ○ Analizar si se obtendrían otros resultados normalizando los datos
Apartado 5	Repetir el análisis de <i>K-means</i> con la librería Kmeans de Scikit-learn y comparar los resultados con la implementación propia

2. Ipython Notebook exportado como html.

3. Código Python (**ficheros .py**) necesario para la correcta ejecución del Notebook. Recuerda que la implementación de *k-means* no necesita seguir el diseño propuesto para los clasificadores y debería estar en un módulo denominado



ClusteringKMeans.py. En el caso de la Regresión Logística se entregará el fichero **ClasificadorRegresionLogistica.py**.

8. Puntuación de cada apartado

La valoración de cada apartado será la siguiente:

- ✓ **Regresión Logística:** 2,5 puntos
- ✓ **Análisis ROC:** 2,5 puntos
- ✓ **K-means:** 3 puntos
- ✓ **Presentación y discusión de los resultados:** 2 puntos